UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA

PROYECTO

Docente:

Jannelly Bello

Alumnos:

Kendra Cabello

Isaac Maluenga

Alejandro Mayol

Enrique Sánchez

Genesis Sánchez

Ciudad Guayana, marzo del 2026

INTRODUCCIÓN

El desarrollo de software requiere aplicar metodologías y técnicas que permitan analizar un problema, diseñar una solución adecuada e implementarla utilizando herramientas de programación. En este contexto, la Programación Orientada a Objetos permite modelar sistemas complejos mediante clases, objetos y relaciones entre ellos, facilitando la organización del código y su mantenimiento.

El presente proyecto tiene como objetivo el desarrollo de una aplicación que simule la dinámica del videojuego **Among Us**, un juego multijugador donde varios participantes interactúan dentro de una nave espacial cumpliendo diferentes roles. Dentro del juego existen dos tipos de jugadores: los tripulantes y los impostores. Mientras los tripulantes deben completar diversas misiones para mantener la nave en funcionamiento, los impostores deben sabotear la nave y eliminar a los tripulantes sin ser descubiertos.

La solución propuesta permitirá simular esta dinámica mediante una aplicación desarrollada en **Java utilizando el IDE NetBeans**, aplicando los principios de la programación orientada a objetos. El sistema permitirá gestionar partidas con múltiples jugadores, asignar roles de forma aleatoria, realizar misiones dentro del mapa, detectar cuando un jugador es eliminado y determinar los ganadores de la partida.

Para lograr este objetivo, se realizará un proceso de desarrollo estructurado que incluye **análisis de requerimientos, diseño del sistema y posterior implementación**, utilizando diagramas UML, casos de uso y modelos de clases que permitan representar el comportamiento del sistema.

CAPÍTULO I

ANÁLISIS

Se identificaron los actores del sistema, las operaciones que deben realizar y las reglas que definirán el funcionamiento de la aplicación.

Este sistema simulará una partida del juego Among Us pero utilizando un mapa inspirado en la UNEG, específicamente la sede de Villa Asia, donde participarán varios jugadores que interactuarán dentro del mapa realizando diferentes acciones. Cada jugador tendrá un rol asignado al inicio de la partida, el cual puede ser tripulante o impostor.

Los tripulantes tienen como objetivo completar las misiones asignadas dentro del mapa o descubrir al impostor mediante votaciones. Por su parte, los impostores deben eliminar a los tripulantes sin ser descubiertos, generando confusión dentro de la partida.

Se deberán considerar diferentes elementos del juego, así como la asignación aleatoria de roles, la realización de misiones, la interacción entre jugadores, los jugadores eliminados y el proceso de votación para identificar a los impostores.

Estos elementos nos permitirán definir los requerimientos funcionales y servirá como base para el diseño del programa.

REQUERIMIENTOS DEL PROGRAMA

Acciones y comportamientos que el programa debe realizar para cumplir con los objetivos del juego.

RF1. Crear partida

El programa debe permitir iniciar una nueva partida del juego.

RF2. Registrar jugadores en la partida

El programa debe permitir incorporar jugadores a la partida con un mínimo de 5 y un máximo de 10 participantes.

RF3. Asignar roles aleatoriamente

El programa debe asignar de forma automática y aleatoria el rol de impostor a uno o más jugadores al iniciar la partida.

RF4. Seleccionar mapa de juego

El programa debe permitir seleccionar entre al menos dos mapas diferentes antes de iniciar la partida.

RF5. Mostrar mapa

El programa debe mostrar el mapa del juego con las diferentes salas disponibles.

RF6. Permitir el desplazamiento entre salas

Los jugadores deben poder desplazarse entre las diferentes áreas del mapa durante la partida.

RF7. Realizar misiones

El programa debe permitir que los jugadores realicen misiones dentro de diferentes salas del mapa. Deben existir al menos cuatro misiones.

RF8. Permitir acciones del impostor

El programa debe permitir que el impostor pueda eliminar o paralizar a los tripulantes cuando se encuentren en contacto.

RF9. Detectar jugadores eliminados

Cuando un jugador sea eliminado por un impostor, el sistema debe marcarlo como inhabilitado dentro del juego.

RF10. Reportar jugadores eliminados

El programa debe permitir que un jugador reporte cuando encuentra a otro jugador eliminado.

RF11. Realizar votaciones

Cuando se convoca una reunión de emergencia, el sistema debe permitir que los jugadores voten para decidir quién es el impostor.

RF12. Expulsar jugador del juego

El programa debe expulsar al jugador seleccionado durante la votación y mostrar si era o no impostor.

RF13. Determinar ganador de la partida

El programa debe evaluar las condiciones del juego y determinar si ganan los tripulantes o los impostores.

RF14. Mostrar resultado final

Al finalizar la partida, el sistema debe mostrar los ganadores.

RF15. Mostrar ayuda del juego

El programa debe incluir una sección de ayuda que explique las reglas y la forma de jugar.

RF16. Mostrar información del sistema

El programa debe incluir una pantalla "Acerca de", donde se muestre información como:

- Lenguaje de programación utilizado
- Librerías utilizadas
- Versión del sistema

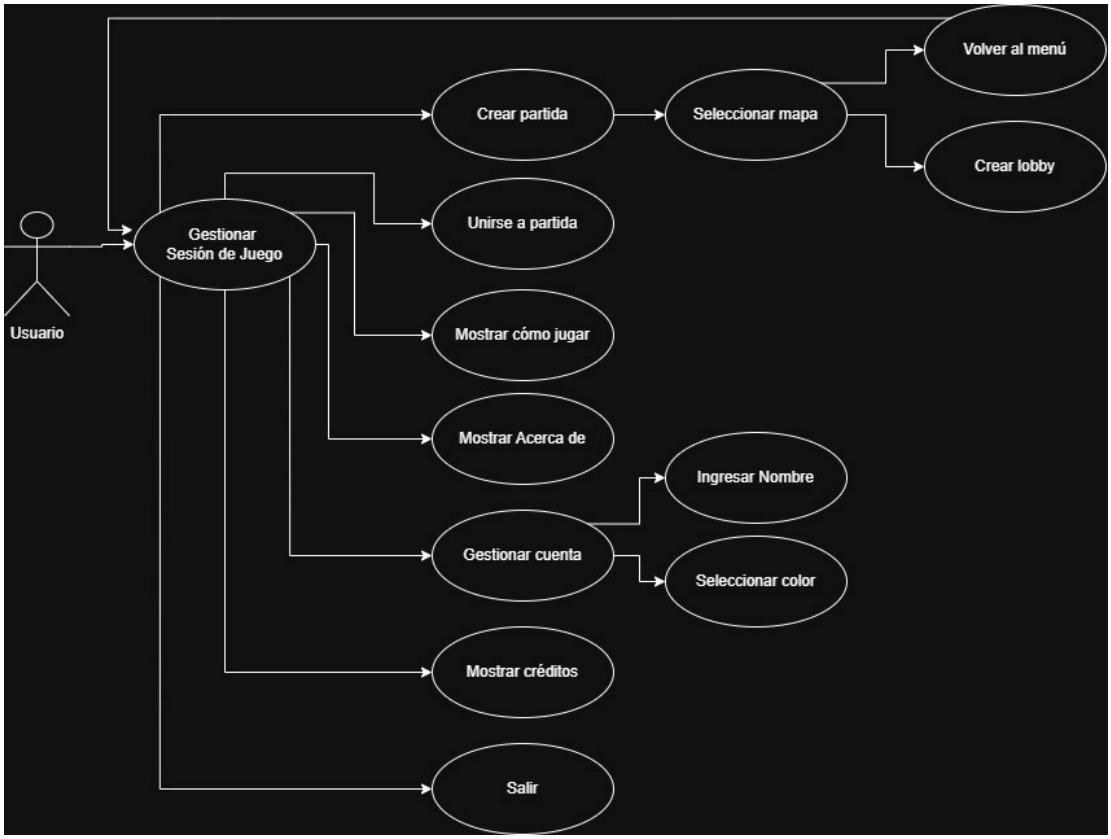
- Desarrolladores del proyecto.

RF17. Almacenamiento de datos

El programa debe permitir almacenar información del juego utilizando archivos XML o archivos propios de Java.

CAPÍTULO II

CASOS DE USO



CASO DE USO	CREAR PARTIDA
ACTORES	Usuario
PROPÓSITO	Permitir al usuario configurar y establecer una nueva sala de juego como anfitrión.
PRECONDICIONES	El usuario debe haber interactuado con el "Gestor sesión de juego".
FLUJO PRINCIPAL	1. El Usuario selecciona la opción a través del menú.

	2. El programa despliega la interfaz de configuración (mapas, número de impostores, etc.). 3. El Usuario confirma los ajustes y el sistema crea la sala.
FLUJO ALTERNATIVO	El programa no logra conectar con el servidor de creación; se muestra un mensaje de error y se regresa al menú principal.

CASO DE USO **UNIRSE A PARTIDA**

ACTORES	Usuario
PROPÓSITO	Acceder a una sesión de juego ya existente mediante un código.
PRECONDICIONES	El usuario debe estar en el menú de juego y el menú debe estar activo.
FLUJO PRINCIPAL	1. El Usuario selecciona "Unirse a partida" desde el Gestor de sesión. 2. El Usuario ingresa un código de sala. 3. El programa valida el acceso y transporta al jugador al lobby.
FLUJO ALTERNATIVO	La sala seleccionada está llena o el código es inválido; el programa notifica al usuario el error.

CASO DE USO **MOSTRAR CÓMO JUGAR**

ACTORES	Usuario
PROPÓSITO	Desplegar las reglas básicas para nuevos jugadores.
PRECONDICIONES	El Gestor de sesión debe estar cargado.

FLUJO PRINCIPAL	1. El Usuario selecciona la opción de “Cómo Jugar”. 2. El programa abre una interfaz con las instrucciones de juego.
FLUJO ALTERNATIVO	El recurso no se muestra correctamente.

CASO DE USO **MOSTRAR ACERCA DE**

ACTORES	Usuario
PROPÓSITO	Visualizar información de lo que se utilizó en la creación del programa.
PRECONDICIONES	Interacción con el Gestor de sesión de juego.
FLUJO PRINCIPAL	1. El Usuario presiona el botón "Acerca de". 2. El programa despliega la ventana con la información.
FLUJO ALTERNATIVO	No aplica flujo de error crítico para esta visualización informativa.

CASO DE USO **GESTIONAR CUENTA**

ACTORES	Usuario
PROPÓSITO	Administrar el perfil del jugador, incluyendo cambios de nombre o cambios de color.
PRECONDICIONES	El usuario debe haber abierto el programa y estar en el flujo de acceso del Gestor.

FLUJO PRINCIPAL	1. El Usuario accede a la sección de cuenta desde el menú principal. 2. El sistema muestra los datos actuales del perfil. 3. El Usuario realiza cambios (ej. cambiar nombre) y guarda los cambios.
FLUJO ALTERNATIVO	Error de asignación y no realiza los cambios.

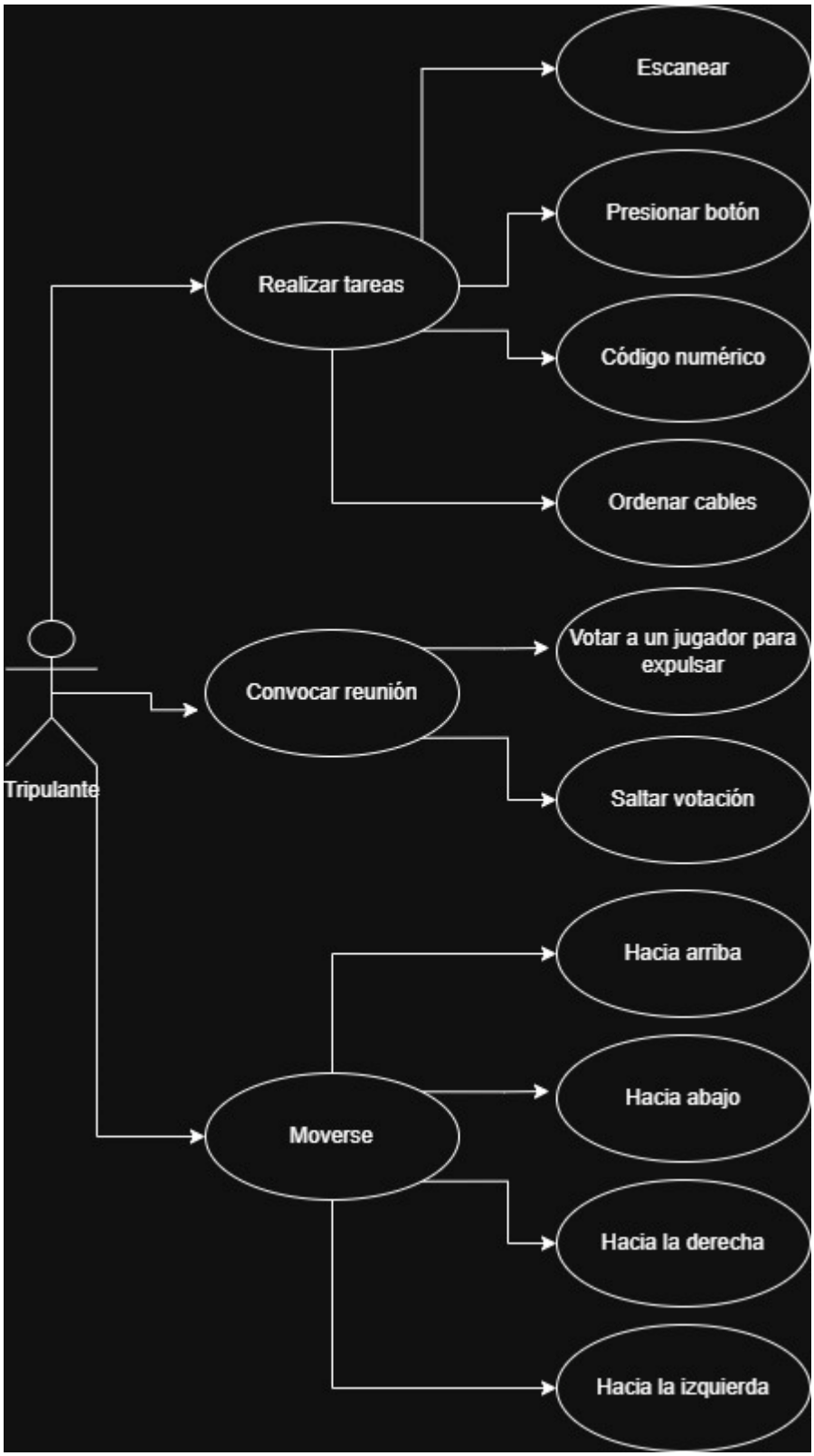
CASO DE USO **MOSTRAR CRÉDITOS**

ACTORES	Usuario
PROPÓSITO	Visualizar información de los participantes de la creación del programa.
PRECONDICIONES	Interacción con el Gestor de sesión.
FLUJO PRINCIPAL	1. El Usuario presiona el botón "Créditos". 2. El programa despliega la ventana con la información.
FLUJO ALTERNATIVO	No aplica flujo de error crítico para esta visualización informativa.

CASO DE USO **SALIR DEL JUEGO**

ACTORES	Usuario
PROPÓSITO	Cerrar de forma segura la aplicación y finalizar la sesión actual.

PRECONDICIONES	El usuario debe estar en el menú principal (fuera de una partida activa).
FLUJO PRINCIPAL	1. El Usuario pulsa el botón de "Salir". 2. El programa se cierra, liberando los recursos del sistema.
FLUJO ALTERNATIVO	No aplica flujo de error crítico.

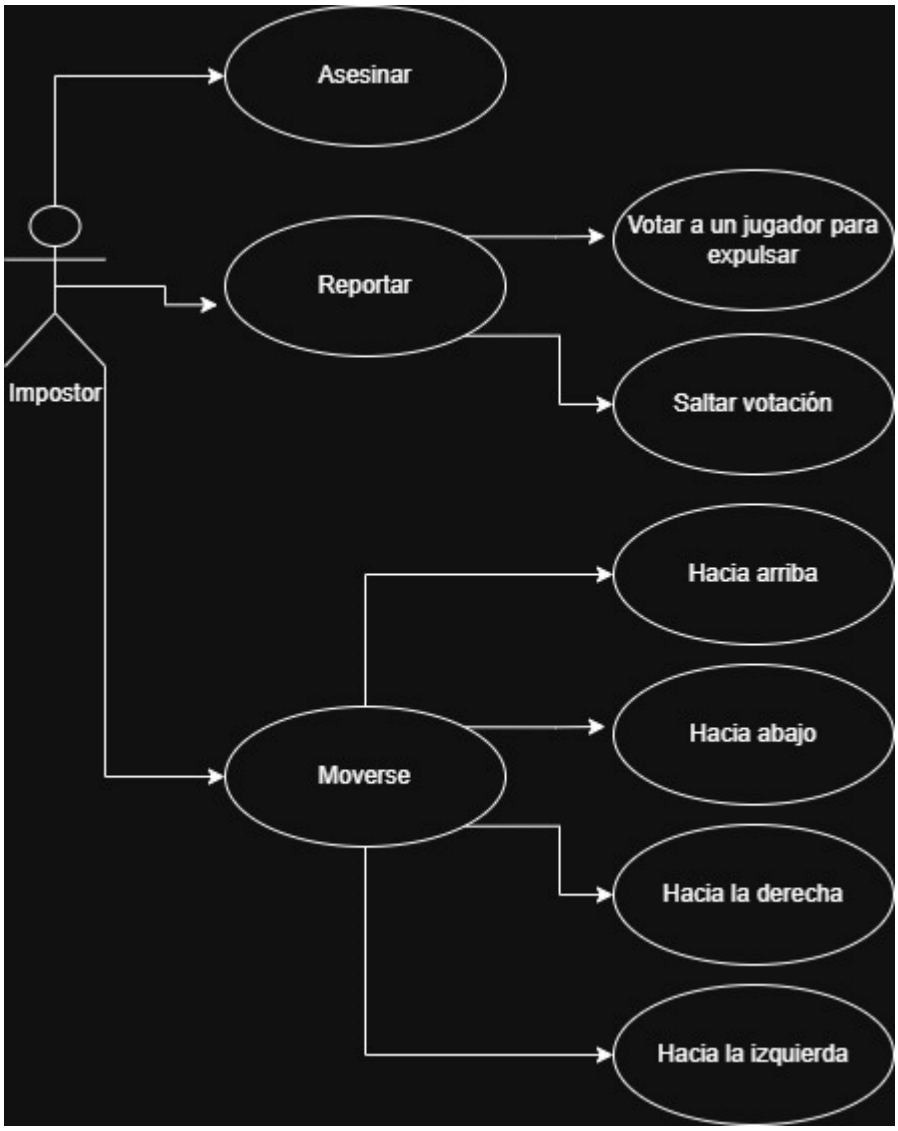

Escanear
Presionar botón
Realizar tareas
Código numérico
Ordenar cables
Votar a un jugador para expulsar
Convocar reunión
Tripulante
Saltar votación
Hacia arriba
Hacia abajo
Moverse
Hacia la derecha
Hacia la izquierda

CASO DE USO	REALIZAR TAREAS
ACTORES	Tripulante
PROPÓSITO	Completar los mini-juegos asignados para llenar la barra de progreso total y ganar la partida.
PRECONDICIONES	El Tripulante debe estar con vida y situado sobre el punto de interacción de la tarea.
FLUJO PRINCIPAL	1. El jugador selecciona "Realizar tareas". 2. El programa despliega una de las interfaces específicas: **Escanear**, **Código numérico**, **Ordenar cables** o **Presionar botón**. 3. El jugador completa la interacción mecánica requerida. 4. El programa cierra la interfaz y suma el progreso a la barra de tareas.
FLUJO ALTERNATIVO	**Interrupción:** Si se reporta un cuerpo la interfaz de la tarea se cierra.

CASO DE USO	CONVOCAR REUNIÓN
ACTORES	Tripulante
PROPÓSITO	Iniciar una reunión de emergencia tras encontrar un cadáver para identificar al impostor.
PRECONDICIONES	El Tripulante debe estar en rango de visión y proximidad de un cuerpo eliminado.
FLUJO PRINCIPAL	1. El jugador presiona el botón "Reportar" en la interfaz. 2. El programa detiene el juego y abre la pantalla de discusión.

	3. Los jugadores proceden a **Votar a un jugador para expulsar** o **Saltar votación**.
FLUJO ALTERNATIVO	**Muerte simultánea:** Si el Tripulante es asesinado en el mismo momento en que intenta reportar, el sistema prioriza la muerte y el cuerpo no es reportado.

CASO DE USO	**MOVERSE**
ACTORES	Tripulante
PROPÓSITO	Desplazar al personaje por las diferentes salas del mapa.
PRECONDICIONES	El personaje debe estar en una zona de juego activa (Lobby o Mapa).
FLUJO PRINCIPAL	1. El jugador utiliza los controles de dirección. 2. El programa procesa la entrada hacia **Arriba**, **Abajo**, **Derecha** o **Izquierda**. 3. El avatar cambia su posición en el eje correspondiente.
FLUJO ALTERNATIVO	**Obstáculo:** El programa detecta una colisión con un muro o mueble y detiene el movimiento en esa dirección específica.



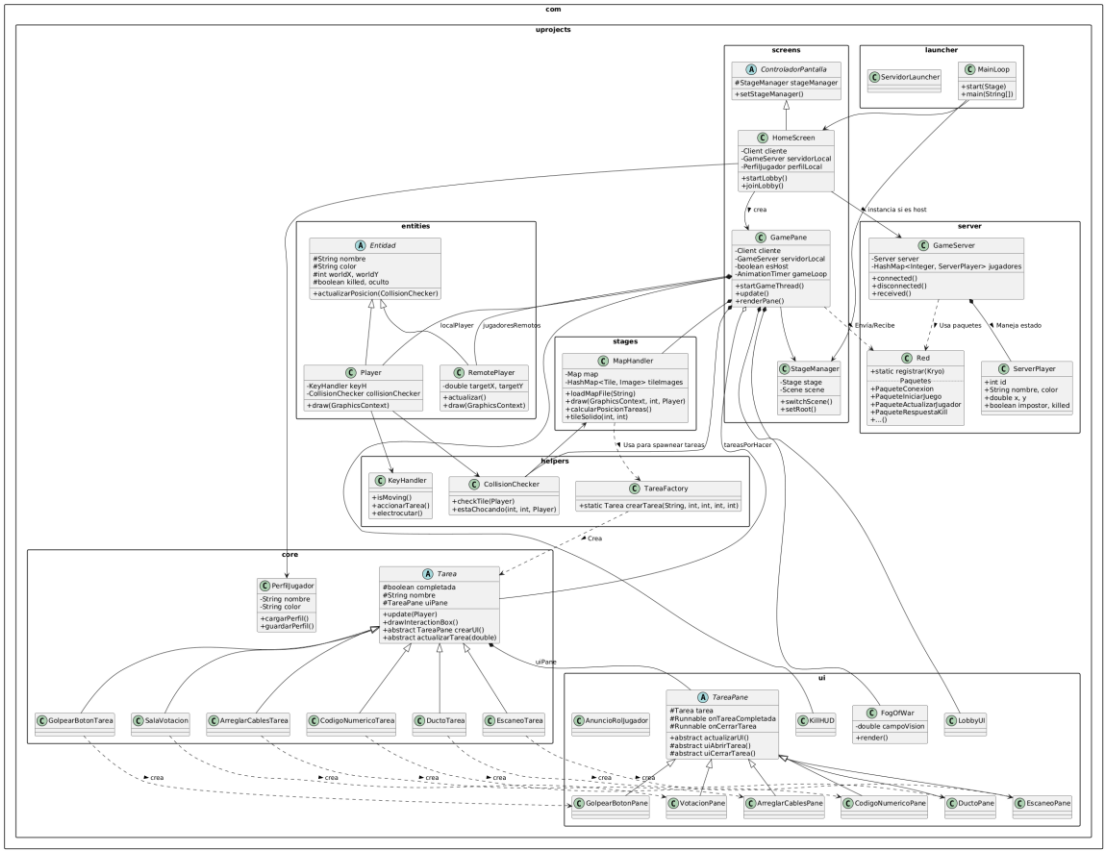
CASO DE USO	**ASESINAR**
ACTORES	**Impostor**
PROPÓSITO	Eliminar a los tripulantes para reducir su número y ganar la partida.
PRECONDICIONES	1. El jugador debe tener el rol de Impostor.

	2. El botón "Kill" debe estar disponible. 3. Un Tripulante debe estar dentro del rango de ataque.
FLUJO PRINCIPAL	1. El Impostor se posiciona cerca de un Tripulante. 2. El actor selecciona la opción **Asesinar**. 3. El programa cambia el estado del Tripulante a "muerto" o “congelado”.
FLUJO ALTERNATIVO	**Objetivo fuera de rango:** Si el Tripulante se aleja justo antes de la acción, el programa no ejecuta la muerte.

CASO DE USO **REPORTAR**

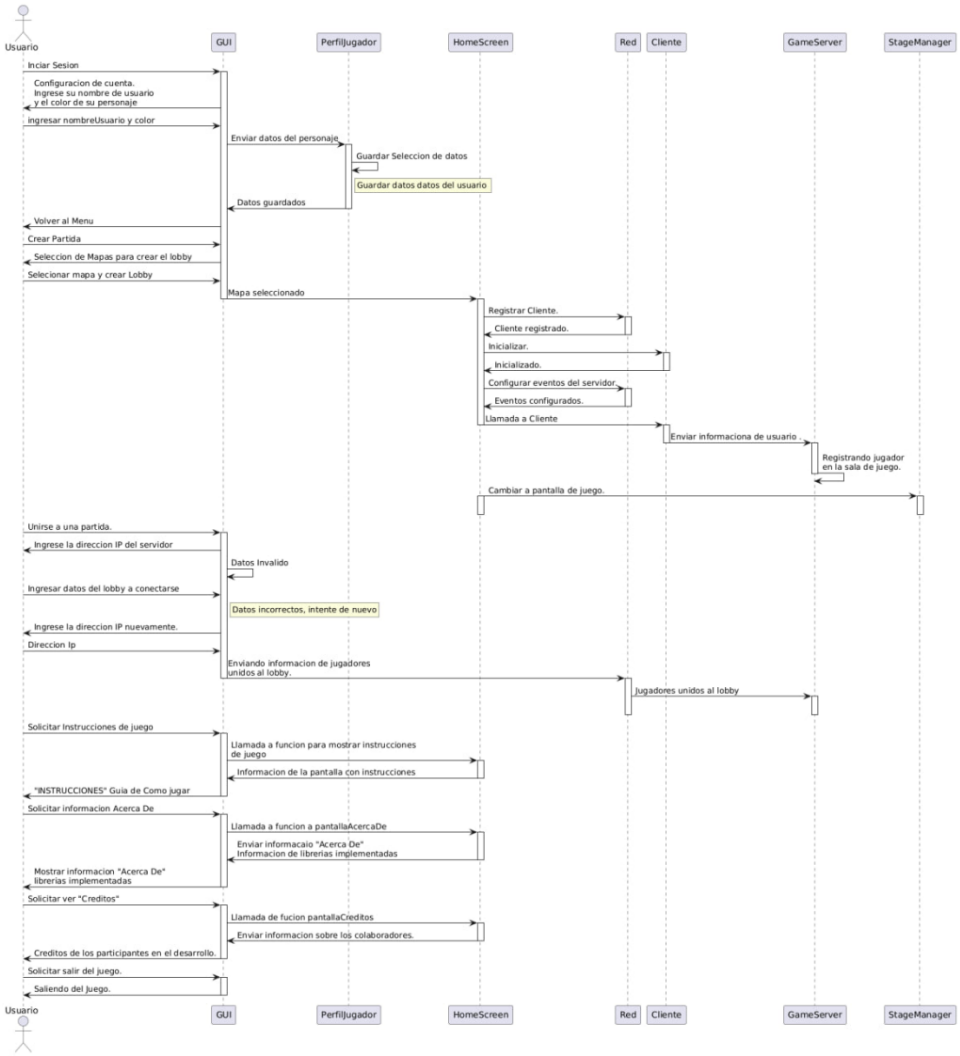
ACTORES	**Impostor**
PROPÓSITO	Informar sobre un cadáver para iniciar una discusión y desviar sospechas hacia otros.
PRECONDICIONES	El Impostor debe estar cerca de un cadáver (propio o ajeno).
FLUJO PRINCIPAL	1. El jugador selecciona la opción **Reportar**. 2. El sistema abre la pantalla de discusión grupal. 3. El jugador puede elegir entre **Votar a un jugador para expulsar** o **Saltar votación**.
FLUJO ALTERNATIVO	**Cuerpo reportado previamente:** Si otro jugador reporta el mismo cuerpo un instante antes, la acción del Impostor se anula y entra directamente a la reunión.

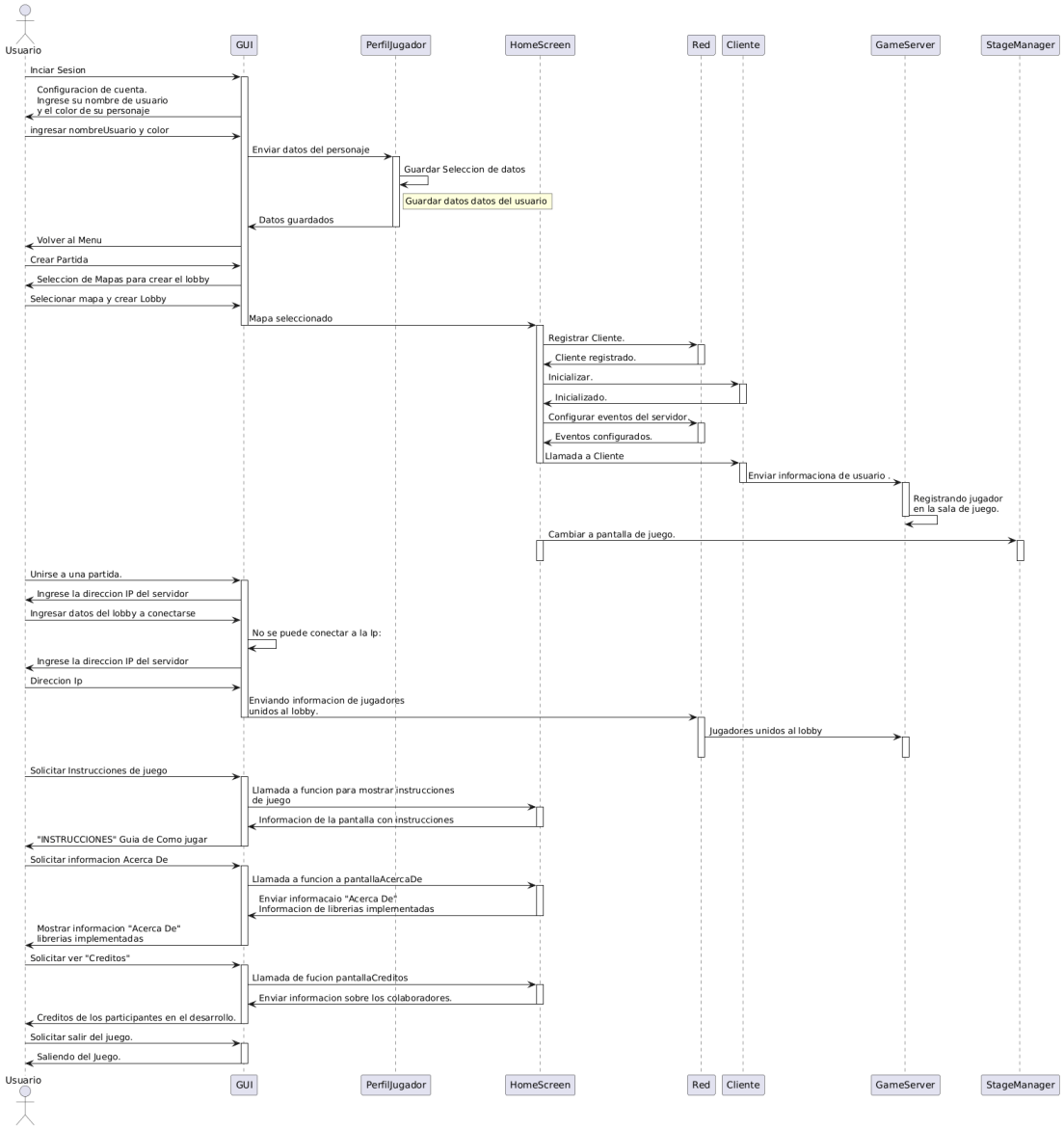
CASO DE USO	MOVERSE
ACTORES	**Impostor**
PROPÓSITO	Desplazarse por el mapa para acechar tripulantes o fingir tareas.
PRECONDICIONES	La partida debe estar en curso y el Impostor no debe estar en una reunión.
FLUJO PRINCIPAL	1. El jugador utiliza los controles para desplazarse. 2. El programa procesa el movimiento **Hacia arriba**, **Hacia abajo**, **Hacia la derecha** o **Hacia la izquierda**. 3. El avatar se actualiza en la posición indicada.
FLUJO ALTERNATIVO	**Colisión con entorno:** El sistema impide el avance si el avatar choca con estructuras fijas del mapa.


com
uprojects
screens
ControladorPantalla
#StageManager stageManager
+setStageManager()
launcher
ServidorLauncher
MainLoop
+start(Stage)
+main(String[])
HomeScreen
-Client cliente
-GameServer servidorLocal
-PerfilJugador perfilLocal
+startLobby()
+joinLobby()
crea
instancia si es host
entities
Entidad
#String nombre
#String color
#int worldX, worldY
#boolean killed, oculto
+actualizarPosicion(CollisionChecker)
GamePane
-Client cliente
-GameServer servidorLocal
-boolean esHost
-AnimationTimer gameLoop
+startGameThread()
+update()
+renderPane()
server
GameServer
-Server server
-HashMap<Integer, ServerPlayer> jugadores
+connected()
+disconnected()
+received()
localPlayer
jugadoresRemotos
Player
-KeyHandler keyH
-CollisionChecker collisionChecker
+draw(GraphicsContext)
RemotePlayer
-double targetX, targetY
+actualizar()
+draw(GraphicsContext)
stages
MapHandler
-Map map
-HashMap<Tile, Image> tileImages
+loadMapFile(String)
+draw(GraphicsContext, int, Player)
+calcularPosicionTareas()
+tieneSolido(int, int)
StageManager
-Stage stage
-Scene scene
+switchScene()
+setRoot()
Envia/Recibe
Usa paquetes
Maneja estado
Red
+static registrar(Kryo)
Paquetes
+PaqueteConexion
+PaqueteIniciarJuego
+PaqueteActualizarJugador
+PaqueteRespuestaKill
+...()
ServerPlayer
+int id
+String nombre, color
+double x, y
+boolean impostor, killed
Usa para spawnear tareas
tareasPorHacer
helpers
KeyHandler
+isMoving()
+accionarTarea()
+electrocutar()
CollisionChecker
+checkTile(Player)
+estaChocando(int, int, Player)
TareaFactory
+static Tarea crearTarea(String, int, int, int, int)
Crea
core
PerfilJugador
-String nombre
-String color
+cargarPerfil()
+guardarPerfil()
Tarea
#boolean completada
#String nombre
#TareaPane uiPane
+update(Player)
+drawInteractionBox()
+abstract TareaPane crearUI()
+abstract actualizarTarea(double)
uiPane
GolpearBotonTarea
SalaVotacion
ArreglarCablesTarea
CodigoNumericoTarea
DuctoTarea
EscaneoTarea
ui
TareaPane
#Tarea tarea
#Runnable onTareaCompletada
#Runnable onCerrarTarea
+abstract actualizarUI()
#abstract uiAbrirTarea()
#abstract uiCerrarTarea()
AnuncioRolJugador
KillHUD
FogOfWar
-double campoVision
+render()
LobbyUI
crea
GolpearBotonPane
VotacionPane
ArreglarCablesPane
CodigoNumericoPane
DuctoPane
EscaneoPane

CAPÍTULO III

DISEÑO




Usuario
GUI
PerfilJugador
HomeScreen
Red
Cliente
GameServer
StageManager
Inciar Sesion
Configuracion de cuenta.
Ingrese su nombre de usuario
y el color de su personaje
ingresar nombreUsuario y color
Enviar datos del personaje
Guardar Seleccion de datos
Guardar datos datos del usuario
Datos guardados
Volver al Menu
Crear Partida
Seleccion de Mapas para crear el lobby
Selecionar mapa y crear Lobby
Mapa seleccionado
Registrar Cliente.
Cliente registrado.
Inicializar.
Inicializado.
Configurar eventos del servidor.
Eventos configurados.
Llamada a Cliente
Enviar informaciona de usuario .
Registrando jugador
en la sala de juego.
Cambiar a pantalla de juego.
Unirse a una partida.
Ingrese la direccion IP del servidor
Ingresar datos del lobby a conectarse
No se puede conectar a la Ip:
Ingrese la direccion IP del servidor
Direccion Ip
Enviando informacion de jugadores
unidos al lobby.
Jugadores unidos al lobby
Solicitar Instrucciones de juego
Llamada a funcion para mostrar instrucciones
de juego
Informacion de la pantalla con instrucciones
"INSTRUCCIONES" Guia de Como jugar
Solicitar informacion Acerca De
Llamada a funcion a pantallaAcercaDe
Enviar informacaio "Acerca De"
Informacion de librerias implementadas
Mostrar informacion "Acerca De"
librerias implementadas
Solicitar ver "Creditos"
Llamada de fucion pantallaCreditos
Enviar informacion sobre los colaboradores.
Creditos de los participantes en el desarrollo.
Solicitar salir del juego.
Saliendo del Juego.